

Systeme de Réservation de Transports

Architecture Logicielle

21 décembre 2025

Résumé

Ce document présente l'architecture et l'implémentation d'un système de réservation de moyens de transport basé sur les principes SOLID de programmation orientée objet. Le système est conçu pour être extensible, maintenable et indépendant des types concrets de transport.

Table des matières

1	Introduction	3
2	Exigences du Système	3
2.1	Description Générale	3
2.2	Calcul du Coût d'une Réservation	3
2.3	Contraintes de Conception	3
3	Architecture du Système	4
3.1	Diagramme de Classes	4
3.2	Structure du Code	4
3.2.1	Interfaces et Protocoles	4
3.2.2	Enumérations	5
3.2.3	Classe de Base	5
4	Implémentation des Classes Concrètes	5
4.1	Voiture	5
4.2	Vélo	6
4.3	Trottinette Électrique	6
5	Système de Réservation	7
5.1	Classe SystemeReservation	7
6	Scénarios de Test	8
6.1	Test 1 : Réservation d'une voiture pour 6 heures	8
6.2	Test 2 : Réservation d'un vélo pour 3 heures	8
6.3	Test 3 : Réservation d'une trottinette avec batterie à 15%	8
6.4	Test Complet	9
7	Extensibilité du Système	10
7.1	Ajout d'un Nouveau Type de Transport	10

8	Principes SOLID Appliqués	10
8.1	Single Responsibility Principle (SRP)	10
8.2	Open/Closed Principle (OCP)	11
8.3	Liskov Substitution Principle (LSP)	11
8.4	Interface Segregation Principle (ISP)	11
8.5	Dependency Inversion Principle (DIP)	11
9	Conclusion	11

1 Introduction

Ce document décrit l'implémentation d'un système de réservation de moyens de transport respectant les principes de conception orientée objet. Le système doit être capable de gérer différents types de transport tout en restant extensible pour l'ajout de nouveaux types sans modification du code existant.

2 Exigences du Système

2.1 Description Générale

Chaque moyen de transport est identifié par :

- **ID** : Identifiant unique
- **Marque** : Fabricant du moyen de transport
- **Tarif horaire de base** : Prix par heure d'utilisation

Le système de réservation doit manipuler uniquement les objets capables d'être réservés, indépendamment de leur type réel. Il ne doit dépendre que des fonctionnalités nécessaires à la réservation et au calcul de coût.

2.2 Calcul du Coût d'une Réservation

Pour toute réservation, le coût total est calculé à partir de la durée (en heures) et peut être ajusté selon des règles spécifiques :

Voitures

- Fonctionnent à l'essence ou diesel
- Une réduction de 10% est appliquée si la durée dépasse 5 heures

Vélos

- Peuvent être équipés d'un système de vitesses
- Le coût dépend uniquement de la durée

Trottinettes électriques

- Possèdent une batterie exprimée en pourcentage
- Si le niveau de batterie est inférieur à 20% avant la réservation, une recharge est obligatoire
- Cette recharge entraîne un coût supplémentaire ajouté au coût total

2.3 Contraintes de Conception

La solution proposée doit :

1. Permettre l'enregistrement de différents moyens de transport
2. Gérer les réservations sans dépendre du type concret du moyen de transport
3. Appliquer automatiquement les règles de calcul appropriées

4. Afficher les informations complètes liées à une réservation
5. Rester flexible, afin que le module de réservation puisse fonctionner avec tout nouvel objet reservable
6. Être extensible, de manière à pouvoir intégrer de nouveaux types de transport sans modifier le code existant

3 Architecture du Système

3.1 Diagramme de Classes

Le système est basé sur le principe de l'Interface Segregation (principe "I" de SOLID) avec l'utilisation d'un protocole pour les objets reservables.



FIGURE 1 – Diagramme de classes du système de réservation

3.2 Structure du Code

3.2.1 Interfaces et Protocoles

```
1 from typing import Protocol
2
```

```

3 class Reservable(Protocol):
4     """Interface pour les objets pouvant être réservés"""
5     def calculer_cout(self, duree_heures: float) -> float: ...
6
7     def get_details_reservation(self, duree_heures: float) -> str: ...

```

Listing 1 – Interface Reservable

3.2.2 Enumérations

```

1 from enum import Enum
2
3 class TypeCarburant(Enum):
4     ESSENCE = "essence"
5     DIESEL = "diesel"
6
7 class SystemeVitesses(Enum):
8     SIMPLE = "simple"
9     MULTIPLE = "multiple"

```

Listing 2 – Énumérations

3.2.3 Classe de Base

```

1 from dataclasses import dataclass
2
3 @dataclass
4 class MoyenTransport:
5     """Classe de base pour tous les moyens de transport"""
6     id: int
7     marque: str
8     tarif_horaire: float

```

Listing 3 – Classe MoyenTransport

4 Implémentation des Classes Concrètes

4.1 Voiture

```

1 class Voiture(MoyenTransport, Reservable):
2     def __init__(self, id: int, marque: str, tarif_horaire: float,
3                 type_carburant: TypeCarburant):
4         super().__init__(id, marque, tarif_horaire)
5         self.type_carburant = type_carburant
6
7     def calculer_cout(self, duree_heures: float) -> float:
8         cout_base = self.tarif_horaire * duree_heures
9
10        if duree_heures > 5:
11            cout_base *= 0.9 # 10% de réduction
12
13        return cout_base
14

```

```

15     def get_details_reservation(self, duree_heures: float) -> str:
16         cout = self.calculer_cout(duree_heures)
17         reduction = "avec r duction de 10%" if duree_heures > 5 else "
18         sans r duction"
19
20         return (f"R servation Voiture: {self.marque} (ID: {self.id})\n"
21                 f"Type carburant: {self.type_carburant.value}\n"
22                 f"Dur e: {duree_heures}h | Tarif horaire: {self.
tarif_horaire} \n"
                f"Co t total: {cout:.2f}      ({reduction})")

```

Listing 4 – Classe Voiture

4.2 Vélo

```

1 class Velo(MoyenTransport, Reservable):
2     def __init__(self, id: int, marque: str, tarif_horaire: float,
3                 systeme_vitesses: SystemeVitesses):
4         super().__init__(id, marque, tarif_horaire)
5         self.systeme_vitesses = systeme_vitesses
6
7     def calculer_cout(self, duree_heures: float) -> float:
8         return self.tarif_horaire * duree_heures
9
10    def get_details_reservation(self, duree_heures: float) -> str:
11        cout = self.calculer_cout(duree_heures)
12
13        return (f"R servation V lo: {self.marque} (ID: {self.id})\n"
14                f"Syst me vitesses: {self.systeme_vitesses.value}\n"
15                f"Dur e: {duree_heures}h | Tarif horaire: {self.
tarif_horaire} \n"
16                f"Co t total: {cout:.2f}  ")

```

Listing 5 – Classe Vélo

4.3 Trottinette Électrique

```

1 class TrottinetteElectrique(MoyenTransport, Reservable):
2     COUT_RECHARGE = 5.0
3
4     def __init__(self, id: int, marque: str, tarif_horaire: float,
5                 niveau_batterie: float):
6         super().__init__(id, marque, tarif_horaire)
7         if not 0 <= niveau_batterie <= 100:
8             raise ValueError("Niveau batterie doit tre entre 0 et 100"
9         )
10        self.niveau_batterie = niveau_batterie
11
12    def calculer_cout(self, duree_heures: float) -> float:
13        cout_base = self.tarif_horaire * duree_heures
14
15        if self.niveau_batterie < 20:
16            cout_base += self.COUT_RECHARGE
17
18        return cout_base

```

```

18
19     def get_details_reservation(self, duree_heures: float) -> str:
20         cout = self.calculer_cout(duree_heures)
21         recharge = "avec recharge obligatoire" if self.niveau_batterie <
20     else "sans recharge"
22
23         return (f"R s ervation Trotinette: {self.marque} (ID: {self.id})
\n"
24                 f"Niveau batterie: {self.niveau_batterie}%\n"
25                 f"Dur e: {duree_heures}h | Tarif horaire: {self.
tarif_horaire} \n"
26                 f"Co t total: {cout:.2f}      ({recharge})")

```

Listing 6 – Classe TrotinetteElectrique

5 Système de Réservation

5.1 Classe SystemeReservation

```

1 class SystemeReservation:
2     """Syst me principal de r s ervation ind pendant des types
concrets"""
3
4     def __init__(self):
5         self.reservations = []
6
7     def enregistrer_reservation(self, transport: Reservable,
duree_heures: float) -> dict:
8         """Enregistre une r s ervation et retourne les d tails"""
9         cout = transport.calculer_cout(duree_heures)
10        details = transport.get_details_reservation(duree_heures)
11
12        reservation = {
13            'transport': transport,
14            'duree': duree_heures,
15            'cout': cout,
16            'details': details
17        }
18
19        self.reservations.append(reservation)
20        return reservation
21
22    def afficher_toutes_reservations(self):
23        """Affiche toutes les r s ervations enregistr es"""
24        if not self.reservations:
25            print("Aucune r s ervation enregistr e.")
26            return
27
28        print("\n" + "="*50)
29        print("LISTE DES R SERVATIONS")
30        print("="*50)
31
32        total_general = 0
33        for i, reservation in enumerate(self.reservations, 1):
34            print(f"\nR s ervation #{i}:")
35            print(reservation['details'])

```

```

36         total_general += reservation['cout']
37
38     print("\n" + "="*50)
39     print(f"TOTAL G N R A L : {total_general:.2f} ")
40     print("="*50)

```

Listing 7 – Classe SystemeReservation

6 Scénarios de Test

6.1 Test 1 : Réservation d'une voiture pour 6 heures

```

1 def test_reservation_voiture():
2     # Cr ation d'une voiture
3     voiture = Voiture(
4         id=1,
5         marque="Toyota",
6         tarif_horaire=20.0,
7         type_carburant=TypeCarburant.ESSENCE
8     )
9
10    # R servation pour 6 heures (doit appliquer r duction)
11    systeme = SystemeReservation()
12    reservation = systeme.enregistrer_reservation(voiture, 6.0)
13
14    print(reservation['details'])
15    # Sortie attendue: Co t total: 108.00 (avec r duction de 10%)

```

Listing 8 – Test Voiture

6.2 Test 2 : Réservation d'un vélo pour 3 heures

```

1 def test_reservation_velo():
2     # Cr ation d'un v lo
3     velo = Velo(
4         id=2,
5         marque="Decathlon",
6         tarif_horaire=5.0,
7         systeme_vitesses=SystemeVitesses.MULTIPLE
8     )
9
10    # R servation pour 3 heures
11    systeme = SystemeReservation()
12    reservation = systeme.enregistrer_reservation(velo, 3.0)
13
14    print(reservation['details'])
15    # Sortie attendue: Co t total: 15.00

```

Listing 9 – Test Vélo

6.3 Test 3 : Réservation d'une trottinette avec batterie à 15%


```

1 def test_reservation_trottinette():
2     # Cr ation d'une trottinette avec batterie faible
3     trottinette = TrottinetteElectrique(
4         id=3,
5         marque="Xiaomi",
6         tarif_horaire=8.0,
7         niveau_batterie=15.0 # < 20%, donc recharge obligatoire
8     )
9
10    # R servation pour 2 heures
11    systeme = SystemeReservation()
12    reservation = systeme.enregistrer_reservation(trottinette, 2.0)
13
14    print(reservation['details'])
15    # Sortie attendue: Co t total: 21.00 (avec recharge obligatoire)

```

Listing 10 – Test Trottinette

6.4 Test Complet

```

1 def test_scenario_complet():
2     """Ex cute tous les sc narios de test"""
3
4     print("="*60)
5     print("SC NARIO COMPLET DE TEST DU SYST ME DE R SERVATION")
6     print("="*60)
7
8     # Initialisation du syst me
9     systeme = SystemeReservation()
10
11    # 1. Voiture pour 6 heures
12    voiture = Voiture(1, "Toyota", 20.0, TypeCarburant.ESSENCE)
13    reservation_voiture = systeme.enregistrer_reservation(voiture, 6.0)
14    print("\n1. " + reservation_voiture['details'])
15
16    # 2. V lo pour 3 heures
17    velo = Velo(2, "Decathlon", 5.0, SystemeVitesses.MULTIPLE)
18    reservation_velo = systeme.enregistrer_reservation(velo, 3.0)
19    print("\n2. " + reservation_velo['details'])
20
21    # 3. Trottinette avec batterie 15%
22    trottinette = TrottinetteElectrique(3, "Xiaomi", 8.0, 15.0)
23    reservation_trottinette = systeme.enregistrer_reservation(
24        trottinette, 2.0)
25    print("\n3. " + reservation_trottinette['details'])
26
27    # Affichage du r capitulatif
28    systeme.afficher_toutes_reservations()
29
30    if __name__ == "__main__":
31        test_scenario_complet()

```

Listing 11 – Test complet intégr 

7 Extensibilité du Système

7.1 Ajout d'un Nouveau Type de Transport

Pour démontrer l'extensibilité du système, voici comment ajouter un nouveau type de transport (ex : Scooter) :

```
1 class Scooter(MoyenTransport, Reservable):
2     """Exemple d'extension du syst me avec un nouveau type de transport
3     """
4     def __init__(self, id: int, marque: str, tarif_horaire: float,
5                 cylindree: int):
6         super().__init__(id, marque, tarif_horaire)
7         self.cylindree = cylindree
8
9     def calculer_cout(self, duree_heures: float) -> float:
10        cout_base = self.tarif_horaire * duree_heures
11
12        # R gle sp cifique aux scooters: tarif r duit pour courte
13        dur e
14        if duree_heures < 2:
15            cout_base *= 0.8 # 20% de r duction
16
17        return cout_base
18
19    def get_details_reservation(self, duree_heures: float) -> str:
20        cout = self.calculer_cout(duree_heures)
21        reduction = "avec r duction de 20%" if duree_heures < 2 else "
22        sans r duction"
23
24        return (f"R servation Scooter: {self.marque} (ID: {self.id})\n"
25                f"Cylindr e: {self.cylindree}cc\n"
26                f"Dur e: {duree_heures}h | Tarif horaire: {self.
27                tarif_horaire} \n"
28                f"Co t total: {cout:.2f}      ({reduction})")
29
30    # Utilisation sans modification du syst me de r servation
31    def test_extension_système():
32        systeme = SystemeReservation()
33
34        # Le syst me fonctionne avec le nouveau type sans modification
35        scooter = Scooter(4, "Honda", 15.0, 125)
36        systeme.enregistrer_reservation(scooter, 1.5)
37
38        systeme.afficher_toutes_reservations()
```

Listing 12 – Nouveau type de transport: Scooter

8 Principes SOLID Appliqués

8.1 Single Responsibility Principle (SRP)

Chaque classe a une seule responsabilité :

- MoyenTransport : stocke les informations de base

- Classes concrètes : implémentent les règles métier spécifiques
- `SystemeReservation` : gère uniquement les réservations

8.2 Open/Closed Principle (OCP)

Le système est ouvert à l'extension mais fermé à la modification. On peut ajouter de nouveaux types de transport sans modifier le code existant.

8.3 Liskov Substitution Principle (LSP)

Tous les types de transport peuvent être substitués à l'interface `Reservable`.

8.4 Interface Segregation Principle (ISP)

L'interface `Reservable` est minimale et spécifique aux besoins de réservation.

8.5 Dependency Inversion Principle (DIP)

Le système de réservation dépend de l'abstraction `Reservable`, pas des implémentations concrètes.

9 Conclusion

L'implémentation présentée respecte toutes les contraintes spécifiées :

- **Enregistrement multiple** : Supporte différents types de transport
- **Indépendance des types** : Utilise le protocole `Reservable`
- **Calcul automatique** : Applique les règles métier spécifiques
- **Affichage complet** : Fournit tous les détails nécessaires
- **Flexibilité** : Fonctionne avec tout nouvel objet reservable
- **Extensibilité** : Permet d'ajouter de nouveaux types sans modification

Le système démontre comment les principes SOLID peuvent être appliqués pour créer une architecture robuste, maintenable et extensible.